Informática II

funciones:

- Declaración y codigo

  Tipo nombre (tipo nom-funcion)

- llamado

  nombre (variable_a_copia)

  Si le paso un puntero es por referencia.

Punteros

- Declaracion Tipo *nombre;
- Asignacion *nombre = a; (valor de la variable)

  nombre = &a; (direccion de la variable

Almacenamiento dinamico

```
#include <malloc.h>

P= (int*) malloc (sizeof (int) * tam);
Variable= (Tipo) malloc (sizeof (tipo) * tamaño deseado del vector);

free(p);  libera espacio.
```

Con new:

```
int *p= new int [tam];
puntero = new tipo [tamaño];

delete [] p;  libera espacio
```

Init a cero malloc

NOTA:

Scanned by CamScanner

PILA (LIFO) la base se va corriendo

```
#include <stdio.h>
#include <stdlib.h>


struct nodo
{
        char dato;
        struct nodo *sig;
};


int main()
{
        struct nodo *p, *base;
        char c;

        p=NULL;
        base=NULL;

        printf("Ingrese caracteres (z para terminar) \n");


        while( (c=getchar()) != 'z' )
        {
                p=(struct nodo*)malloc(sizeof(*p));

                if(p!=NULL)
                {       (*p).dato=c;
                        (*p).sig=base;
                        base=p;
                }
                else
                {       printf("No se pudo gestionar la memoria \n");
                        return 1;
                }
        }

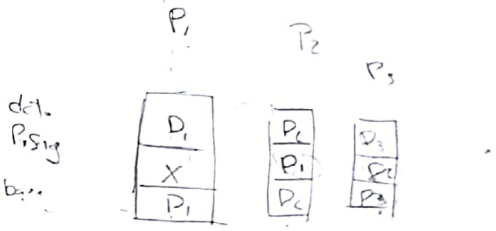

        /-----RECORRERLA------/
        printf("\nLa lista ingresada es la siguiente \n\n");
        p=base;
        while(p!=NULL)
        {
                printf("%c", (*p).dato);
                p=(*p).sig;
        }

        system("PAUSE");
        return 0;
}
```

Funciona

1) creo P
2) Almacena
3) P.sig = base
4) Base = P
5) nuevo P

while



Scanned by CamScanner

COLA (FIFO) base = de fue

Funciona

1) Creo P
2) Base = P
3) Almacena el dato en P
4) P.sig = null
5) Creo q
6) P.sig = q
7) q.dato = dato (Almacena en q)
8) q.sig = null
9) P = q

while

```
#include<stdio.h>
#include<stdlib.h>


struct nodo
{       int dato;
        struct nodo *sig;
};
int getint(void);


int main()
{       struct nodo *p, *q, *base;
        int n;
        p=NULL;             q=NULL;             base=NULL;

        p=(struct nodo*)malloc(sizeof(*p));
        if(p==NULL)
        {       printf("No hay memoria\n");
                return 1;
        }
        else
        {       printf("Comienze a ingresar los datos (0 para terminar) \n");

                        base=p;
                        printf("Ingrese un entero: ");
                        n=getint();
                        (*p).dato=n;
                        (*p).sig=NULL;
                        printf("Ingrese un entero :");

                        while( (n=getint())!=0)
                        {       q=(struct nodo*)malloc(sizeof(*q));
                                if(q==NULL)
                                {       printf("no hay memoria\n");
                                        return 1;
                                }
                                else
                                {               (*p).sig=q;
                                                (*q).dato=n;
                                                (*q).sig=NULL;
                                                p=q;
                                        printf("Ingrese un entero: ");
                                }
                        }
        }
```



Scanned by CamScanner

Recorrer

```
        printf("\nLos datos ingresados son:\n");

        p=base;
        while(p!=NULL)
        {       printf("%d\n", (*p).dato);
                p=(*p).sig;
        }


        free(base);     free(p);                free(q);

        system("PAUSE");
        return 0;
}


int getint(void)
{
        int n;
                scanf("%d", &n);
        return n;
}
```

Scanned by CamScanner

LISTA DOBLE

```
#include <stdio.h>
#include <stdlib.h>


struct nodo
        { char dato;
        struct nodo *sig;
        struct nodo *ant;
        };

int main()
{       struct nodo *p,*q,*base1,*base2;
        char c;
        p=q=base1=base2=NULL;

        printf("Ingresa una letra ('s' para salir)\n");

        p=(struct nodo*)malloc(sizeof(*p));
        base1=p;
        base2=p;
        c=getchar();
        (*p).dato=c;
        (*p).sig=NULL;
        (*p).ant=NULL;

        while((c=getchar() )!='s')
        {       q=(struct nodo*)malloc(sizeof(*q));
                (*p).sig=q;
                (*q).ant=p;
                p=q;
                base2=q;
                (*p).dato=c;
        }

        //------  Recorrido COLA-------
        printf("\nRecorrido en forma de cola\n");
        p=base1;
        while(p!=NULL)
        {       printf("%c",(*p).dato);
                p=(*p).sig;
        }

        //------  Recorrido PILA-------
        printf("\nRecorrido en forma de pila\n");
        p=base2;
        while(p!=NULL)
        {       printf("%c",(*p).dato);
                p=(*p).ant;
        }


return 0;       }
```

Pasos:

1) Creo P
2) Base 1, 2 = P
3) Almacena datos P
4) P.sig = P.ant = null
5) Creo q
6) P.sig = q
7) q.ant = P
8) P = q
9) base 2 = q
10) Almacena dato en P.

Scanned by CamScanner

RECURSION

1) Implementar la serie de fibonacci en forma recursiva y no recursiva

```
#include <stdlib.h>
#include <stdio.h>
int fibonaci(int num);

int main()
{       int n;
        int resultado;


        scanf("%d",&n);
        resultado=fibonaci(n);
        printf("%d", resultado );

return 0;
}



int fibonaci(int num)
{
        if(num==0 || num==1) return num;
        else return fibonaci(num-1)+fibonaci(num-2);
}
```

2) Implementar mediante un método recursivo, el cálculo de factorial de un número entero. Implementar la versión no recursiva.

```
int factorial(int num);
int main()
{       int n;
        scanf("%d",&n);


        printf("%d", (factorial(n)) );
return 0;
}



int factorial(int num)
{       if(num==0) return 1;
        else return(num *factorial(num-1));
}
```

Scanned by CamScanner

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <malloc.h>
//using namespace std;


struct nodo{
                int dato;
struct nodo *pder;
struct nodo *pizq;
};


void inorden(struct nodo *p);
void posorden(struct nodo *p);
void preorden(struct nodo *p);


int main(){
        struct nodo *nuevo,*raiz,*actual,*marcador;
int num;
raiz=(struct nodo *)malloc(sizeof(*raiz));
printf("Ingrese el dato de la raiz \n");
scanf("%d",&num);
raiz->dato=num;
raiz->pder=NULL;
raiz->pizq=NULL;
while(num!=0)
{marcador=raiz;
while(marcador!=NULL)
{actual=marcador;
                                                if (num> (actual->dato))
                                                        {marcador=actual->pder;}
                                                else
                                                        {marcador=actual->pizq;}
}
                nuevo=(struct nodo *)malloc(sizeof(*nuevo));
        nuevo->dato=num;
                nuevo->pder=NULL;
                nuevo->pizq=NULL;
                        if(num> (actual->dato))
                                actual->pder=nuevo;
                        else
                                actual->pizq=nuevo;
printf("Ingrese el dato del nodo \n");
scanf("%d",&num);
}; //Cierro el while
```

Scanned by CamScanner

```
printf("Recorrido inorden(de menor a mayor) \n");
inorden(raiz);
printf("Recorrido posorden(Primero izq,luego der,luego raiz) \n");
posorden(raiz);
printf("Recorrido preorden(primero raiz,luego izq y ultimo derecha) \n");
preorden(raiz);
printf("\n");
system("pause");
return 0;
}
void inorden(struct nodo *p)
{if (p!=NULL)
                        {inorden(p->pizq);
printf ("%d",p->dato);
printf("\n");
inorden(p->pder);
}
}
void preorden(struct nodo *p)
{if (p!=NULL)
{printf ("%d",p->dato);
printf("\n");
                        preorden(p->pizq);
preorden(p->pder);
}
}
void posorden(struct nodo *p)
{if (p!=NULL)
{posorden(p->pizq);
posorden(p->pder);
printf ("%d",p->dato);
printf("\n");
}
}
```

Scanned by CamScanner

1 byte = 8 o 9 bits

el PS envía los datos bit a bit por el mismo hilo

Puerto Paralelo (PP) (cable ≤ 3m)

- Se usa para conectar una PC clásica a una impresora, se la sigue utilizando por su facilidad de manejo

- Post (power on self test) identifica los PP disponibles y escribe sus direcciones en una zona del RAM

- los bits de datos viajan juntos, enviando paquetes de byte a la vez

- Bus: cableado q' conecta la PC con los otros periféricos

- los SO basados en "DOS", gestionan las interfaces de pp con los nombres LPT1, LPT2, así sucesivamente
En cambio en "Unix" los nombran como /dev/lp0, /dev/lp1

las direcciones base de los primeros puertos son

	LPT1	LPT2	LPT3
Dato	3BCh	378h	278h
Estado	3BDh	379h	279h
Control	3BEh	37Ah	27Ah

- Tiene 3 registros: corresponden a las 3 funciones q' podemos esperar del manejo de la comunicación a través de un puerto

1) Registro de control:

- Bit 7, 6, 5 no se usa.
- Bit 4: IRQ (Interrupt Request) Genera interrupción cuando ACK=0 no está conectado a los pines de salida (no se usa)
General mente las impresoras trabajan por "polling" es decir un loop q' comprueba la condición.
- Bit 3: Select In (en 1 → impresora On line
en 0 → " Off line
- Bit 2: Init inicia reset de impresora (cuando envía un 0 la PC a la impresora)
- Bit 1: Auto feed (carriage return)
algunas impresoras requieren un retorno de carro adicional q' se logra con un LF (line feed) después de un CR
- Bit 0: Strobe: al enviar un dato (líneas D0 a D7) la impre. se pone a 0 er inmediatamente se vuelve a 1 esperando el dato sig.

NOTA:

Scanned by CamScanner

Para inicializar la impresora en un programa o p/enviar un dato ponemos en el registro de control online (3) y reset (2) en 1

```
outport (0x37A , 0x0C);
outport (0x378 , dato);
```

// Strobe
// dato

outport (direcc del puerto, dato)
↳ se usa para enviar datos

B7	B6	B5	B4	B3	B2	B1	B0
—	—	—	IRQ	Select in	Init	Auto feed	Strobe

↳ es el bit mas importante o significativo

2) Registro de estado

los ocho bits estan asociados a las siguientes funciones.

=0 ▲ Bit 7: Busy (0: impresora ocupada Pin 11) trabaja en logica negativa de modo q' al estar seteado = 0

la impresora requiere un tiempo para procesar el caracter y envia un Busy inmediatamente luego de un Strobe. El procesador debe esperar la desactivacion de Busy para enviar el caracter sig.

=0 ▲ Bit 6: ACK (Acknowledgment) confirma la recepcion del ultimo caracter → 1 significa q' la impr. recibio un caracter

=1 ▲ Bit 5: PE (Peper eneable) detecta si falta papel (PE = 0)

=1 ▲ Bit 4: SLCT on indica q' la impresora se encuentra conectada y lista

=0 ▲ Bit 3: Error (error de transf de datos en la transmision)

▲ Bit 2, 1 no se usa

=1 ▲ Bit 0: Time out error lapso de tiempo para establecer la conexion a partir del cual se presume q' hay algo un error q' lo impide

B7	B6	B5	B4	B3	B2	B1	B0
Busy	Ack	PC	Slct on	Error	-	-	time out error

↳ bit mas importante

NOTA:

Scanned by CamScanner

3) Registro de dato (envía dato)

Se usa la orden "output (puerto, dato)

Buss: cableado q conecta la CPU con otro dispositivo

líneas:

1) Potencia (canal de alimentación)
2) Entrada / salida o lectura / escritura (I/O ; R/W)
3) Mover datos (datos)
4) Direcciones (al enviar un dato no sabemos q se va a parar a otro lado, por eso llegamos con dirección)

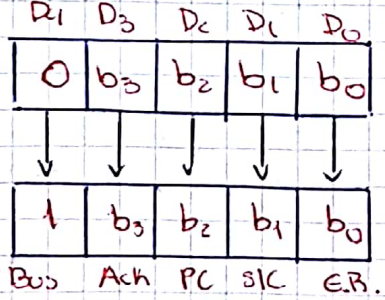
Red → concepto básico es escribir y leer

Nibbles Primero se transmite el inferior después el superior

▲ Comunicación a través del puerto paralelo.

El PP se puede utilizar como un canal de comunicación de propósito general, sin embargo, al no poder controlar el envío de un byte completo, es necesario enviar c/byte separando en dos mitades (nibbles) y reunirlo en el receptor.

▲ Paso 1: el emisor transmite el nibble inferior, colocando un 0 en D_4 para q aparezca un 1 en la señal Busy de destino

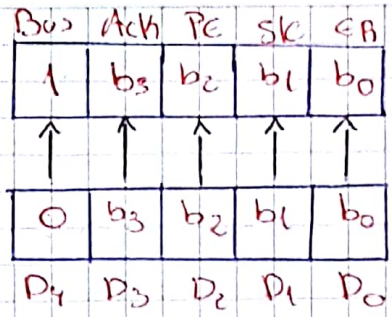


▲ Paso 2: El receptor ha estado esperando el cambio de la señal Busy de 0 a 1, para poder leer el nibble inferior ubicado en las líneas D.0 - D_3

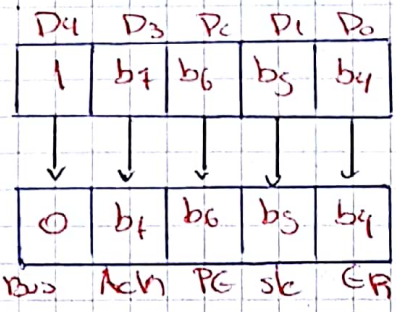
Cuando esto pasa coloca el nibble en sus líneas D_0 - D_3 y coloca D_4 en 0

NOTA:

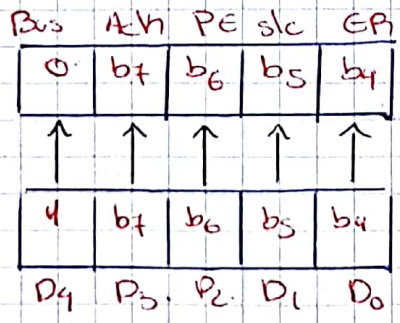
Scanned by CamScanner



▲ Paso 3: el emisor ahora envía en nibble superior, llevando el bit D4 a 1 de esta manera el receptor leerá la entrada cuando reciba un cero en Busy



▲ Paso 4: Esta vez el receptor ha esperado q' el bit Busy cambie a 0. El nibble ahora recibido es nuevamente retornado al Transmisor para su comprobación y D4 es opuesto a 1 El Transmisor encuentra así su bit de Busy puesto a 0



la transferencia del byte ha q'dado completa, el receptor reensambla el byte a partir de los 2 nibbles recibidos y el transmisor chequea q' no hubo error

El chequeo puede realizarse en el paso 3 pero generalmente se hace una vez recibido el byte completo

Interrupciones:

la interrupción 17H provee 3 funciones para el manejo del puerto

- ▲ 00H: imprimir caracter
- ▲ 01H: inicializar impresora
- ▲ 02H: reportar estado de impresora

NOTA:

Scanned by CamScanner

Puerto Serie (PS) (cable < 8m)

- los sistemas en serie transmiten la información mediante un bit por vez. la comunicación serial se utilizan en la mayoría de los dispositivos de red, terminales, modems y teclados

- En las comunicaciones seriales c/ byte ~~los enviados o recibidos~~ q' se envía o recibe es enviado de a un bit por vez
  Cada bit puede estar
  - Activado (on) 'mark'
  - Desactivado (off) 'space'

- la velocidad de transmisión se expresa en bits por segundo (bps) o Baudios,

RS232

- Es una interfaz eléctrica estándar p/ comunicaciones seriales. Existen variantes (A, B y C) q' definen distintos rangos de tensión para los niveles on y off

- La mas común es RS-232 C q' define un bit:

  on (mark) → tensiones entre -3v y -12v
  off (space) → " " +3v y +12v

- El RS232 define 18 señales diferentes para la comunicación serial. las mas importantes son:

Tierra
- GND: (logic Ground) Tierra lógica no es una señal, pero sin ella las demas no son operativas. trabaja como una tensión de referencia de modo q' la electrónica pueda saber q' tensiones son (+) o (-)

Transmite datos
- TXD: (Transmitted data) transporta los bits a transmitidos desde la PC. hacia otro dispositivo.
  Una tensión mark = 1 lógico
  space = 0 lógico

Recibe datos
- RXD: (received data) transporta los bits q' recibe la PC

detecta la portadora
- DCD (data carrier detected) una señal de "space" en esta línea indica q' la PC esta conectada correctamente

Terminal de datos listo
- DTR (data terminal Ready) informa al otro dispositivo q' se esta listo (space) o no. usualmente es habilitada automaticamente cuando se inicia la interface serial

equipo de datos listo
- DSR (data set ready) informa q' el conjunto de caracteres transmitido ya ha sido procesado y se esta listo para transmitir nuevos caracteres

NOTA:

Scanned by CamScanner

solicitud enviar

- RTS (request to send) para ser enviado. "space" significa q' mas datos estan listos

disponible para enviar

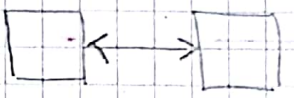
- CTS (clear to send) enviar mas datos al otro extremo "space" indica q' todo esta correcto para

Modos de transmisión

- Full duplex: - significa q' el computador puede enviar y recibir datos simultaneamente
  - Puede ser por 2 vias o por multiplexación de frecuencias



- Half duplex: transmite en ambos sentidos pero no en forma simultanea



- Simplex transmite en un solo sentido

flow control

handshaking (reconocimiento)

Es para regular el flujo de datos ~~de~~ ~~entre~~ al transferir datos entre 2 interfaces seriales. Hay 2 metodos

- Software handshaking: usa caracteres de control especiales para comenzar la transmision (XON o DC1, 021 octal) y para parar (XOff o DC3, 023 octal) el flujo de datos
  Sirven para transmitir informacion textual no pueden ser empleados para transmitir otro tipo de informacion sin una programacion especial
- Hardware handshaking: emplea las señales CTS y RTS entres de caracteres especiales.
  - El receptor setea CTS a tension "space" cuando esta listo para recibir mas datos y a "mark" cuando no lo esta
  - Del mismo modo el transmisor setea RTS cuando esta listo para transmitir.

  Es mas rapido, para' no tiene q' enviar y procesar bits adicionales

Break (error)

- Normalmente una señal de recepcion o transmision permanece en mark hasta q' se transmite un nuevo caracter
- Si la señal se mantiene en space por un periodo largo (1/2 o 1/4 segundos) se supone una condicion de error, entonces se dice q' hay un "Break" en la ~~conexion~~ transmision

NOTA:

Scanned by CamScanner

Modos de Conexión

Para evitar q' ambos equipos intenten transmitir simultaneamente por la misma linea, estos se dividen en:

- DTE (data terminal equipment) ej PC
- DCE (data communication equipment) ej modem

DTE controla a DCE

las formas en q' pueden conectarse son las siguientes

1) Comunicacion de una via (2 o 3 y 7)

DTE		DCE	
2	⟶	2	TxD
7	———	7	GND

Solo uno se transmite y uno se recibe

2) Comunicacion de una via con doble handshaking DTE a DCE

DTE		DCE	
2	⟶	2	TxD
5	⟵	5	CTS
6	⟵	6	DSR
7	———	7	GND

la tension en la linea 6 es mantenida (+) para poder enviar la info

3) Comunicacion de una via con handshaking DCE a DTE

DTE		DCE	
3	⟵	3	TxD
4	⟶	4	RTS
20	⟶	20	DTR
7	———	7	GND

4) Comunicacion de 2 vias

DTE		DCE		
2	⟶	2	TxD	datos
3	⟵	3	RxD	datos
4	⟶	4	RTS	~
5	⟵	5	CTS	~
6	⟵	6	DRS	~
7	———	7	GND	
8	⟵	8	RI	ring indicador
20	⟶	20	DTR	~
22	⟵	22	CD: carrier detect	

~: handshaking

El DTE envia la solicitud de envio (RTS) a lo cual el modem contesta con un aviso de q' esta listo para enviar (CTS). Este handshaking es basicamente para confirmar la conexion.
El DTE agrega el aviso de listo para enviar (DTR), a lo cual el modem contesta con un aviso de q' esta listo para recibir (DSR)

NOTA:

Scanned by CamScanner

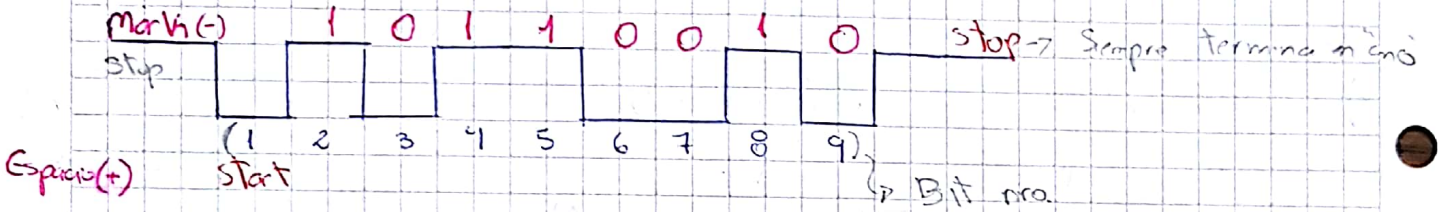
5) Conexion Null Modem o modem nulo - DB9

DTE		DTE
2	→	3
3	←	2
4	→	5
5	←	4
6	←	20
20	→	6
7	—	7

En este caso simplemente se cruzan los canales de Transmisión / recepción y handshaking

Parametros de transmision y Trama (frame)

Cada caracter se transmite dentro de un paquete de usualmente 10 bits.



Para separar los caracteres y sincronizar la transmision del dato se dispone de un bit de arranque (start) y un bit final (stop)

Despues de un bit de Stop viene un bit de Start.

Debe tener:

Bit de arranque		Se produce cuando hay un Space o voltaje (+)
Bit de datos	5, 6, 7, 8	va de 5 a 8, 6 mas comun sin 7,8
Paridad	N, O, E	
Bit(s) de parada	1, 1.5, 2	

Paridad:

N = sin paridad
O = impar
E = par
S = (space) siempre "0"
M = (marca) siempre "1"

Par = cantidad de "1" par
Impar = cantidad de "1" impar

Es una forma sencilla de verificar si hay errores en la transmision serial

Velocidad

	Modems.	Null modem
a 2400 lo	2400	50000 a 100000 + bps
multiplico por 2	4800	
y el resultado	9600	
por 2 y	19200	
asi	38400	
	56000	

NOTA:

Scanned by CamScanner

Transmision Sincrona y Asincrona

- Sincrona: Se transmite un grupo de bytes (mensaje) en forma secuencial, despues del bloque de sincronizacion.
  En general es mas rapida q la asincrona.
  Requiere calidad en el control de temporizacion
- Asincrona: Sincroniza byte por byte, es la q se utiliza mayormente en las PC. No es critica en temporizacion. Si esta falla se produce un error de trama.
  - Es sincrona por caracteres.

Registros de la UART (es una placa)

Se ocupa de recibir datos en paralelo y entregarlos en serie y viceversa.
Tambien dispone de todo lo necesario para operar bajo el estandar RS-232

Tiene 3 registros:

- Datos (mantiene caracteres entrantes y salientes en la transmision)
  - → Buffer de entrada
  - → Buffer de salida
- Estado (informa a la CPU el estado de la UART)
  - → Estado de la linea
  - → Estado del modem
  - → Estado de las interrupciones
- Control (recibe comandos de la CPU)
  - → Control de la linea
  - → Control del modem
  - → Control de interrupciones
  - → Control del cerrojo

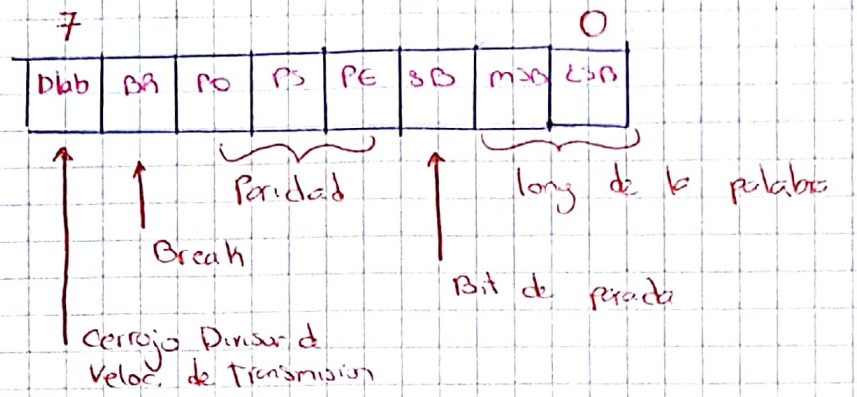
- En total son 9 registros

NOTA:

Scanned by CamScanner

Registro de Control de línea.

Se usa para especificar los parámetros de comunicación



DLAB: siempre en 1, permite el acceso al divisor de la frecuencia fija de reloj para seleccionar la velocidad de transmisión

Break: =1; coloca un Break en la línea de transmisión

Paridad:

sin	00 o 10
par	11
impar	01

Bit de parada

uno	0
dos	1

long de la palabra

siete	10
ocho	11

NOTA:

Scanned by CamScanner

1) ¿Cuáles son la diferencia en la transmisión de datos en serie y paralelo? ventajas y desventajas de cada uno de ellos.

- La principal diferencia entre una conexión serie y una paralelo, es que en la conexión serie los bits se transmiten uno detrás del otro en cambio en el paralelo se transmiten tramas de 8 bits a la vez.
- La conexión paralela es mucho más rápida que la serie y es empleada para la transmisión en distancias cortas.
- Una ventaja significativa de la transmisión serial en relación a la paralela es un menor costo del cableado puesto que se necesita un solo cable se tiene un octavo del costo que se ocuparía para transmisión paralela.

2) ¿Cuál es la diferencia entre la transmisión serie síncrona y asíncrona?

- **Síncrona:** Se transmite un grupo de bytes (mensaje) en forma secuencial. Antes de iniciar la comunicación ambos dispositivos deben de establecer una sincronización entre ellos. Para esto, antes de enviar los datos se envían un grupo de caracteres especiales de sincronía. Una vez que se logra la sincronía, se pueden empezar a transmitir datos. En general es más rápida que la asíncrona. Requiere calidad en el control de temporización, por lo que poseen un hardware más sofisticado.
- **Asíncrona:** Sincroniza byte por byte, es la que se utiliza mayoritariamente en las PCs. No es crítica en temporización. Si esta falla se produce un error de trama.

3) Según la norma rs232c ¿cuál es la máxima distancia a la que pueden conectarse dos equipos?

- La máxima distancia de conexión es 15 mts.

4) ¿Cuáles son los niveles de tensión de los valores lógicos espacio y marca 0 y 1 y la corriente máxima de cortocircuito?

El Puerto Serie toma como 1 (valor lógico alto) a cualquier voltaje que se encuentre entre –3 y –25 V y como 0 (bajo nivel), a tensiones que se encuentren entre +3 y +25 V, es decir:

- Un "Espacio" (0 lógico) estará entre +3 y +25 V. +12 v
- Una "Marca" (1 lógico) estará entre -3 y -25 V. -12 v

La corriente de cortocircuito no debe exceder los **500mA**.

5) De algunos eje de uso de dispositivos que utilizan la transmisión de datos en serie.

- Redes PC- PC, módems, dispositivos USB.

6) ¿A qué nivel lógico corresponde el estado de reposo en la línea de transmisión?

- A un nivel 1 logico.

7) ¿Qué es un baudio, porque se llama así? ¿y qué es el baud rate?

Scanned by CamScanner

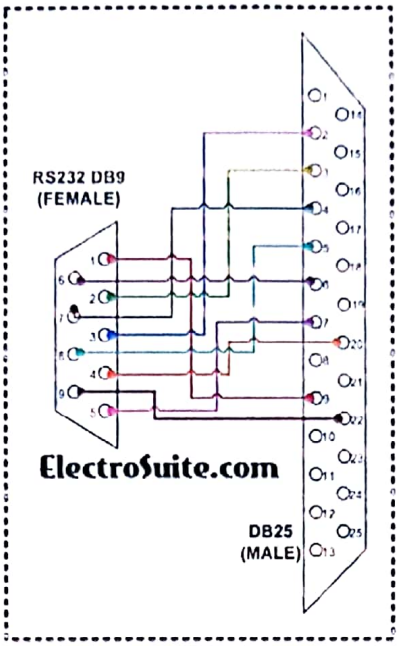
- El **baudio** es una unidad de medida que determina el número de cambios de estados en una señal por segundo. En la transmisión de datos, baudio es la cantidad de veces que cambia el estado del medio de transmisión por segundo. Cada cambio de estado afecta a más de un bit de información, por lo tanto la tasa de bits puede ser superior a la tase de baudios. Esta unidad fue utilizada originariamente para medir la velocidad de las transmisiones telegráficas, tomando su nombre del ingeniero francés **Jean Maurice Baudot**, quien fue el primero en realizar este tipo de mediciones.
- **Baud rate** es la cantidad de baudios por unidad de tiempo.

i) ¿Cuáles son los pines de conexión de un conector estándar DB25 y un DB9? ¿cuales se usan en la práctica y que funciones cumplen cada uno?
   - DB25: posee 25 pines
   - DB9: posee 9 pines, es más utilizado que el DB25 para transmisión por el puerto serie.

Número de clavija	Nombre
2	TXD: Transmitir datos
3	RXD: Recibir datos
4	RTS: Permiso para transmitir
5	CTS: Listo para enviar
6	DSR: Ajuste de datos listo
7	GND: Señal de tierra
8	CD: Detector de transmisión
20	DTR: Terminal de datos lista
22	RI: Indicador de llamada

Scanned by CamScanner

2) Realice un esquema de un adaptador entre ambos conectores



3) ¿Cómo es la secuencia de transmisión de los bits?

- La secuencia de transmisión de los bits es uno detrás del otro, de allí deriva su nombre.

4) ¿Qué funciones cumple la UART? ¿cuáles son sus registros?

- **UART** son las siglas de "**U**niversal **A**synchronous **R**eceiver-**T**ransmitter" (en español, *Transmisor-Receptor Asíncrono Universal*). Este controla los puertos y dispositivos serie. Se encuentra integrado en la placa base o en la tarjeta adaptadora del dispositivo.
- Sus registros son:

  Line control register: Se usa para especificar los parámetros de comunicaciones.

  Modem control register: Controla los reconocimientos con el modem.

  Interrupt Enable Register: Si se habilitan las interrupciones, este registro indica a la UART los eventos habilitados.

  Line Status Register: Se usa para obtener información de los datos de transmición y recepción.

  Modem Status Register: Mediante este registro conocemos el estado de las líneas externas que conectan al modem.

  Interrupt Identificacion Register: Cuando se produce una interrupción, en este registro se busca la causa.

5) Explique una conexión null-modem y para qué sirve

- Es necesario cuando se necesita conectar dos DTE entre sí. Consiste en cruzar la transmisión con la recepción y los respectivos hand-shakings permitiendo que cada uno de los extremos crea que tiene un DCE en el otro.

Scanned by CamScanner

6) ¿Cales son las velocidades típicas en las que se producen las transmisiones? ¿Qué ocurriría si u transmisor emite a diferentes baudios a lo esperado por el receptor?
   - Las velocidades típicas son (en baudios): 2400, 4800, 9600, 19200, 38400, 56000.
   - Ocurriría un error de trama.

7) ¿Cuál es la máxima capacidad de cable seria en pF?
   La máxima capacidad es 2500 pF.
8) Método de control del flujo de datos por soft y hard control de errores y compresión de datos.

Control de flujo

En una conexión hay implicados seis vínculos distintos: equipo de transmisión a módem de transmisión, módem de transmisión a módem de recepción, módem de recepción a equipo y el recorrido inverso de los tres vínculos. Todos ellos pueden tener distintas velocidades de transmisión de datos. Cuando el módem de recepción no puede aceptar datos temporalmente, necesita una forma de indicar al módem de transmisión que vaya más despacio o que espere. El control de flujo es el método por el que un módem controla la velocidad a la que los restantes módems le envían los datos.

Si el control de flujo no está configurado correctamente, puede que le resulte imposible conectarse a un sistema remoto, que la velocidad de transferencia disminuya considerablemente o que se interrumpa la conexión. Si ve muchos errores y reintentos de transmisión de datos al descargar los archivos, compruebe la configuración del control de flujo del módem y del programa de comunicaciones. La configuración del control de flujo del programa de comunicaciones y del módem debe ser la misma. Muchos programas de comunicaciones establecen esta configuración automáticamente en función del módem, aunque algunos deben ser configurados por separado.

Control de flujo por Soft

El control de flujo por software (llamado también XON/XOFF o CTRL+S/CTRL+Q) utiliza caracteres de datos para indicar que el flujo de datos debe iniciarse o detenerse. Esto permite a un módem enviar un carácter de control para indicar a otro módem que detenga la transmisión mientras se actualiza.

El control de flujo por software es más lento y, normalmente, menos conveniente que el control de flujo por hardware. El control de flujo por software se utiliza sólo para transmitir texto. No se puede utilizar para la transferencia de archivos binarios porque éstos pueden contener caracteres especiales de control de flujo.

Control de flujo por Hard

El control de flujo por hardware (RTS/CTS) depende del módem para controlar el flujo de datos. Se debe usar con todos los módems de alta velocidad o con los módems que comprimen datos.

Control de Errores

Corrección de errores es una técnica de módem utilizada para asegurarse de que los bloques de datos son recibidos por el módem remoto con precisión. Las combinaciones comunes de corrección de errores son MNP 4 y V.42.

El proceso funciona de esta forma: primero se recopilan datos que se transmite en un bloque de caracteres. Un algoritmo genera un número basado en el bloque (una suma de comprobación) y lo asocia a continuación al final del bloque. Cuando el otro módem recibió el bloque, ejecuta el mismo algoritmo y compara el resultado con la suma enviada de comprobación. Si estos valores no coinciden, el módem de recibirse indica al remitente si retransmite el bloque.

Scanned by CamScanner

A causa de que se puede tener que retransmitir un bloque de datos, los módem almacenan bloques en un búfer hasta seguro para que el módem de recibirse lo haya procesado correctamente. El módem debe utilizar control de flujo para mantener que el DTE sobrescribe el marco con datos nuevos. Porque RAS no responde a control de flujo, puede sobrescribir un bloque que aún no se ha confirmado.

AsyBEUI que es casi idéntico a NetBEUI implementa su propio control de errores. Todos los tipos LLC los marcos se deben reconocer explícitamente o implícitamente en una cantidad determinada de tiempo o AsyBEUI los retransmite 2 (conexión orientada). Si corrección de errores de módem retransmite un marco entonces supera el tiempo de espera y entonces retransmite el marco, de nuevo, la duplicación de tráfico innecesario ralentiza cosas y generalmente confunde a RAS.

La razón única para activar corrección de errores con RAS es utilizar compresión de datos (que requiere la mayor parte de módems corrección de errores de compresión de datos). Si desea utilizar corrección de errores, consulte el extracto del archivo LEAME.TXT al final de este artículo por favor. Además debería hacer lo siguiente:

• En cliente y servidor, active control de flujo de hardware en su módem. De nuevo, RAS omite esto pero no interferirá al menos con transferencia de datos. Agregue el comando de control de flujo al comando INIT del archivo MODEMS.INF.

• En cliente y servidor, active corrección de errores en el módem. Utilice v.42 si está disponible. Agregue el comando AT a la línea de archivo MODEMS.INF COMANDO =. Consulte el manual RAS para más información sobre la edición de MODEMS.INF

Compresión de datos:

La compresión de datos permite que la información se transmita a una velocidad superior a la velocidad de conexión real. Normalmente, los datos y, en particular, el texto y los gráficos, contienen secuencias repetidas de información idéntica. La compresión de datos funciona al sustituir muchos caracteres de información repetida por unos pocos caracteres y transmitir sólo una copia de las secuencias de datos repetidas.

Los métodos más comunes de compresión de datos son MNP 3, MNP 4, MNP 5 y V.42bis.

La efectividad de la compresión varía en función de los datos. A menudo, el texto y los gráficos se pueden comprimir considerablemente. La compresión de los datos que ya están comprimidos puede ser menos eficiente porque el módem intenta comprimir datos que ya lo están. V.42bis trata los datos comprimidos con mayor eficacia que el resto de los protocolos.

Para que funcione cualquier protocolo de corrección de errores o de compresión de datos, debe ser compatible con los módems de ambos extremos de la conexión. Cuando se conectan dos módems, negocian automáticamente para determinar el mejor protocolo mutuo. Por ejemplo, si falla V.42bis, el de reemplazo es MNP 5 y, si éste falla, el siguiente es "sin compresión".

Para conseguir la máxima velocidad en la transferencia de datos, se requiere el control de flujo por hardware y que la velocidad máxima del puerto se establezca como de dos a cuatro veces la velocidad nominal del módem. Normalmente, estos valores se configuran correctamente cuando se instala el módem. Si sabe qué tipo de compresión admite el módem, es posible calcular este valor multiplicando la velocidad del módem por la estimación indicada anteriormente. Por ejemplo, para un módem de 14,4 kilobits por segundo (Kbps) con compresión V.42bis, deberá realizar el siguiente cálculo:

14,4 Kbps x 4 = 57,6 Kbps

Este valor es la velocidad máxima a la que el módem puede transferir los datos. El rendimiento real depende de los datos.

Scanned by CamScanner

Compresión de datos de software

El software de comunicaciones, como Conexiones de red, puede admitir compresión de datos. Por ejemplo, con un módem V.90 de 56 Kbps, puede habilitar la compresión por software y conseguir un rendimiento medio de 44 Kbps. Las pruebas muestran que la compresión por software puede dar como resultado una velocidad de transferencia mayor que la compresión por hardware.

Si utiliza compresión por software, puede ser aconsejable comprobar la configuración. Para obtener mejores resultados con la compresión por software, puede comprobar las siguientes propiedades del módem:

Habilitar la corrección de errores del módem si se configura Protocolo de datos como EC estándar o EC forzado.

Deshabilitar la compresión por hardware si se configura Compresión como Desactivada.

Habilitar la compresión por software proporcionada por el software de comunicaciones.

9) ¿Cómo funciona el control de paridad?

- Son tres bits que definen las cinco combinaciones posibles de paridad. El más significativo (Parity One) está en 1 cuando la paridad es fija (Mark o Space), el que le sigue (Parity Select) indica la clase de paridad seleccionada, el menos significativo (Parity Enable) está en 1 cuando se adopta alguna clase de paridad.

10) ¿Cómo se establece la frecuencia de transmisión de la UART?

- Se establece mediante dos registros: El DLL y el DLM

11) ¿Cuánto tiempo requerirá una transmisión si tuviera que enviar 64 caracteres de 7 bits a 9600 baudios con un bit de arranque y uno stop?

- 0,00666 segundos.

12) ¿Que establece el teorema del muestreo de Nysquit?

- El teorema demuestra que la reconstrucción exacta de una señal periódica continua en banda base a partir de sus muestras, es matemáticamente posible si la señal está limitada en banda y la tasa de muestreo es superior al doble de su ancho de banda.

13) ¿Qué es el ancho de banda en un sistema de transmisión?

- Es la cantidad de información o de datos que se puede enviar a través de una conexión de red en un período dado.

Scanned by CamScanner

Campos de Bit y uniones

Es un metodo predefinido para acceder a un unico bit en un byte

Declaración:

```
Struct  nomb_estr
  {       tipo - nombre 1 : longitud;
          tipo - nombre 2 : longitud;
  }
```

Si la mayoria de los compiladores admiten campos signed, unsigned, int se debe trabajar con unsigned

Ej.

```
struct byte
  {
      unsigned a8:1
        "      a7:1
        "      a6:1
        "      a5:1
        "      a4:1
        "      a3:1
        "      a2:1
        "      a1:1
  }
```

- Los bits de un campo de bits se asignan igual q' los elementos de una estructura, utilizando el punto si es una variable comun o el operador flecha si es un apuntador a escribe puntero

- No se puede averiguar la direccion de un campo de bit
  Ej: &mystruct.x no anda
  x es c.bit

- No se puede construir arreglos de un campo de bit

- los campos de bit se utilizan para empaquetar mas variables en menor espacio, pero requieren q' el compilador genere codigo adicional para su manipulación. la cual es mas costoso en tiempo de ejecución

NOTA:

Scanned by CamScanner

Operadores con campos de bits

▲ Activar un bit sin afectar al resto (setear un bit)

```
cbit= cbit | Estebit
```

```
  01100100   cbits
OR 00010000   Estebit
   01110100   Resultado
```

▲ Desactivar un bit sin afectar al resto (poner bit a 0)

```
cbit= cbit & ~Estebit
```

```
  01011110   cbit
AND 11101111   Estebit
   01001110   resultado
```

convertir los 0 en 1 o los 1 en 0

▲ Determinar si un bit esta en 1

```
if(stado & 16)
  { printf("1");
  }
```

```
  00110100   stado
  00010000   16
  00010000   resultado
```

Al ser el resultado "0", no esta seteado ese bit

```
  00110100   stado
  00010000   16
  00010000   resultado
```

Al haber un 1, significa q' esta seteado ese bit

▲ Operadores de corrimiento

b b>>4 (se desplaza)

```
b    = 11010000
b>>4 = 00001101
```

Tambien se logra multiplicando o dividiendo por potencias de 2

NOTA:

Scanned by CamScanner

Uniones

- Es una posición de memoria q' es compartida por 2 o más variables diferentes, generalmente de distintos tipo y en distintos momentos
- Su definición es similar a la de una estructura

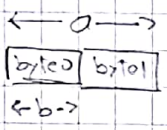
Definición

```
union ejemplo
{
    int a;
    char b;
};

int main()
{ union ejemplo c; }
```

También se pueden hacer arreglos

Comparten de la sig. forma



Al declarar una unión el compilador crea una área de almacenamiento de igual tamaño a la más grande de las variables declaradas.

Enumeraciones

creo q' no va, está en pag 17

NOTA:

Scanned by CamScanner

Shell y Kernel

Shell → la capa visible al usuario, interpreta ordenes q' recibe y las Transmite al Kernel mediante Systems Calls

- shell estandar — /bin/sh
- c-shell — /bin/csh
- korn shell — /bin/ksh
- bourne Again shell — /bin/bash → Utilizamos y viene x default

Permite ejecutar comandos, explorar el arbol de directorios, crear editar y eliminar archivos, etc

Kernel → Se comunica con el hardware. No Tiene ninguna vinculacion con el usuario

funciones: Administrar de procesos
" de dispositivos

Interaccion Kernel y Shell

cp — copia lo al contrario (cp hi.txt 1.txi) (cp hi.txt descargas)

mkdir — (mkdir hiihe) crea una carpeta

rm dir → rmdir hiihe.txt (borra archivo)
rmdir hiihe (borrar carpeta)

Procesamiento de Archivos

Mostrar → cat (cat hiihe.txt) muestra el archivo de una vez
- more =
- nl — lineas — 1 - toda
- pr — Paginas
- lpr =
- less — muestra el archivo pantalla a pantalla
- head =
- tail =

Tamaño → ls
→ wc — n° de lineas - palabras - bytes

Mover y Renombrar → mv (mv hiihe lu) (mv hiihe descargas) lo mueve

Ordenar → sort (numeros / letras (todo / solo el primer digito)) (-r (inverso), -f (ignore m y M) - y (orden n°))

? busca archivo → find

Comparar → diff (diff hi.txt 1.txi) muestra las diferencias
si son iguales no lo muestra

NOTA:

Scanned by CamScanner

buscar palabra en un archivo → grep (grep '...' [ruta] [ubicación])

listado de propiedades de archivos

más común
(ls␣-l) o ls␣-l (lista los detalles de los archivos uno por renglón

drwxr-xr-x 2 kirbe kirbe 4096 nov 22 11:51 descargas

1) Permisos 2) Contador de Vínculos 3) ID del Usuario 4) Ubicación 5) tamaño 6) fecha de Creación 7) Nombre

ls␣-il → muestra lo mismo q' ls␣-l pero unos n° adelante de todo. ↳ se llama n° de inodo (i-nodo)

Tipos de Archivos → Unix, todo son archivos (su clave del éxito)

Son todos para con i-nodo

- Archivos Ordinarios (-)
  ↳ su contenido es interpretado por los programas de Aplicación

  Tienen:
  - un nombre (no necesariamente único)
  - un número único dentro de file system llamado (i-nodo)
  - tamaño en bytes
  - Fecha - hora del último cambio, modificación y acceso
  - Permisos de acceso
  - un dueño y grupo al cual pertenece

- Archivos especiales de dispositivo (b) (c)

  Cada dispositivo físico del sistema (disco externo, impresoras, etc) son asignados a un "archivo especial" el q' a pesar de tener un tratamiento particular es visto por el sistema como un archivo común

  (en ubicación dos ini ttt)

  Ej b rwx---r-x

- Archivos de directorio (d)

  son los lugares donde se guarda conceptualmente, no físicamente los archivos de cualquiera de las 3 clases

Ej "drwxr-xr-x"

- Otros Tipos: c -> de dispositivos de caracteres (impresoras)
  l -> de enlace simbólico

NOTA:

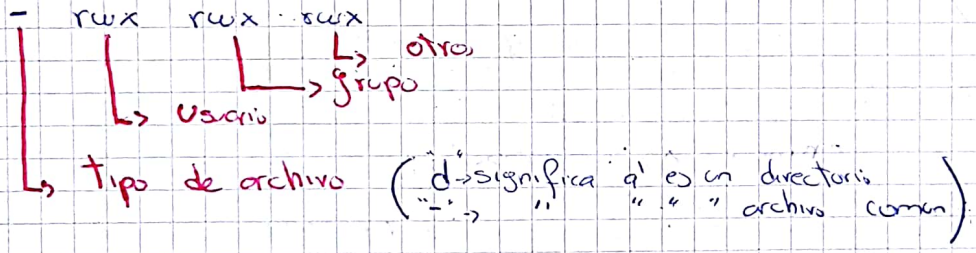
Scanned by CamScanner

Protección y Permisos de Archivos

- los archivos son protegidos por permisos de acceso.

- (r) read → P/ ordinarios → todo se puede ver el archivo no modificarlo
  P/ directorios → permite leer el contenido del directorio, se necesita el permiso 'x'
- (w) write → P/ ordinarios → permite modificar o borrar un archivo
  P/ directorios → permite crear, modificar, o borrar un archivo de cualquier tipo
- (x) execute → P/ ordinarios → si el archivo es un programa permite ejecutarlo
  P/ directorios → si falta no se puede iniciar por el directorio en cuestión

- Entidades:
  - (U) → user (permisos para el dueño del archivo o directorio)
  - (G) → Group (grupo al cual pertenece el archivo)
  - (O) → others (permisos de acceso para todos los demás)



- Para cambiar permisos:
  - chmod → lo realiza el dueño del archivo, o root

2 formas simbólica, numérica

- Simbólica: -rwxr-xr-x;

chmod u-x, g+w, o+rw "nombre" → este comando le elimina la ejecución para usuario, coloca el bit de escritura para el grupo y le da a "todos" el acceso de lectura y escritura

-rw-rwx-rw

Asignarle a todos (a = all)

chmod a+x archivo = chmod +x archivo

NOTA:

Scanned by CamScanner

- Asignarle a todos (a=all)

chmod a+x archivo = chmod +x archivo (le asigna a todos el permiso de ejecución)

- Asignarle permisos específicos

chmod ug=rwx archivo (le asigna a usuario y grupo esos permisos)

- Copia permisos

chmod o=g archivo (copia los permisos de g y se los asigna a otros)

▲ Numerica

los bits r, w, x tienen asignado un nº, q' viene de la notación binaria q' se utiliza para almacenarlos como bits

$\begin{cases} r = 4 \\ w = 2 \\ x = 1 \end{cases}$ se los puede sumar:

rwx	= 7
r-x	= 5
-wx	= 3
--x	= 1
rw-	= 6
r--	= 4
-w-	= 2
---	= 0

el codigo:

chmod xyz 'nombre'
- x = usuario
- y = grupo
- z = otros

Ej: chmod 644 'nombre'
- 6 = 4+2 = rw- (permisos para U)
- 4 = r-- (" " G)
- 4 = r-- (" " O)

-rw-r--r--

hoja a sgte

▲ Para cambiar propietario y grupo

Se necesita ser el usuario propietario o root (sudo -i)

Ojito

chown -> cambia al propietario | Ej: ls -l /etc/host
drwx------ 6 root root 9 nov. /etc/hosts
usuario

chgrp -> cambia el grupo | chown kike /etc/hosts

Se pueden cambiar (cambiar el propietario y el grupo)

chown kike.linux 'nombre'
- kike = propietario
- linux = grupo

NOTA:

Scanned by CamScanner

Lista de Archivos : Usuarios y Grupos

/etc/passwd : Información reservada de las cuentas de usuario (determina quien puede acceder al sistema de manera legítima y q' se puede hacer una vez dentro del sistema; este archivo es la primera línea de defensa contra accesos indeseados)

$ cat /etc/passwd

usuario1 : x : 500 : 500 : usuario nihe : /home/usuario1 : /bin/bash
1 2 3 4 5 6 7

1) nombre de la cuenta (login)
2) clave de acceso encriptada (password) — *: X: significa está almacenada en shadow; !: cuenta bloqueada; !!: no tiene
3) UID (numero unico, identificador del usuario) — 0 pertenece al adm (root); +500 reservado para los usuarios del sistema; -500 reservado para el sistema.
4) GID (identificador numerico unico del grupo del usuario)
5) Nombre del usuario (Gecos - es un campo opcional q' contiene información adicional del usuario)
6) Directorio de trabajo ($ Home)
7) Shell (por defecto /bin/bash)

/etc/shadow (sudo cat /etc/shadow) se almacenan las contraseñas cifradas y nos da información sobre caducidad y validez de la cuenta

usuario1 : 1 ej xtZ4 1$ UHNQ had : 1181 : 0 : : : : : 0
1 2 3 4 5

1) nombre de usuario
2) Password Encriptado
3) Dias desde el 1 Enero de 1970 q' el password fue cambiado
4) Dias antes de q' el password pueda ser cambiado
5) es un campo adicional opcional, generalmente se deja en blanco y representa los dias antes de q' el password expire en el, q' el usuario es advertido.

NOTA:

Scanned by CamScanner

/etc/group : Almacena la información de grupos o define los grupos de usuario
↳ define los grupos al cual pertenece los usuarios

daemon : x : 24 : root, bin, daemon
1 2 3 4

1) nombre del grupo
2) Password encriptado del grupo (se suele no usar, se deja en blanco)
3) ID de grupo (GID)
4) lista de grupo (usuarios pertenecientes al grupo, separados por coma)

Enlaces (link)

los enlaces a archivos permiten verlos desde distintos lugares.
Hay 2 clases → comunes
→ simbolicos

hyperlink

Para cambiar Propietario y Grupo

chown { nombre ubicación
chgrp {

Se necesita ser root (sudo -e)

chown (cambia Propietario) usuario grupo
Eg # ls -l /etc/hosts
-rw-r--r-- 1 root root 9 nov /etc/hosts
chown hilke /etc/hosts
-rw-r--r-- 1 hilke root 9 nov /etc/hosts

chgrp (cambia grupo)

Eg lo mismo de arriba por

chgrp hilke /etc/hosts
-rw-r--r-- 1 root hilke 9 nov /etc/hosts

Chown hilke.hilke /etc/hosts cambia usuario y grupo al mismo tiempo

Al modificar el usuario / permisos / grupo de un archivo, se hacen efectivo al iniciar de nuevo sección.
*) modificar el prop o grupo no afecta al archivo, ya q existe otra forma de hacerlo mediante chmod

Scanned by CamScanner

Herramientas para desarrollo de software

gcc → "gcc hihe.c" obtenemos un archivo ejecutable "a.out" para correr ese ejecutable escribimos "./a.out"

- -o (ponerle nombre al ejecutable) "gcc hihe.c -o hihegempl." (nombre del ejecutable)
- ./ (ejecutarlo) "./hihegempl"
- -c (crea un archivo objeto) "gcc hihe.c -c" (y aparecen .o)
- gcc tambien funciona como enlazador de archivos dependientes

1) hihe1.c

```
int suma (int, int);

int main()
{   int a=5, b=5;
    Printf ("la suma es", suma(a,b));
}
```

2) hihe2.c

```
int suma (int a, int b)
{ return (a+b); }
```

gcc -c hihe1.c hihe2.c crea archivos .o

gcc hihe1.o hihe2.o -o suma (nombre) crea el ejecutable

./suma lo ejecuto y obtengo los archivos combinados

NOTA:

Scanned by CamScanner

Pestaña mayus + ctrl + t

Copia shift + ctrl + c
Pega " " + v

FECHA 18
HOJA:

Prompt

si termina
nom@maquina ~ $ → ($) = conectado como usuario exit
↳ (#) = conectado " root Sudo -i

Arbol de directorios

ls / ...

- bin — Contiene los ejecutables de las ordenes de linux
- boot — " el software de bodeo
- der — controladores de dispositivos perifericos
- home — contiene el directorio del usuario
- lib — " librerias C, C++
- Tmp — " archivos temporales; q' se borran periodicamente
- etc — " " especificos para la instalacion
- usr — " compiladores (/bin ; / gcc)

Visualizar directorio Actual

~$ Pwd (muestra el directorio actual)
~$ echo ~ (el tilde tambien representa el directorio actual)

Comando de consola

--help
man ... (ej man dir) manual de dir

dir (muestra las carpeta q' tenemos)
cd (cambia el directorio) ej (cd Escritorio) (cd "solo" va hacia home)

Trucos

△ Con tab completamos

ctrl + l → limpiar pantalla
ctrl + a → ir al inicio de la linea
ctrl + e → " " fin " " "

ctrl + shift + v => pegar

whoami : devuelve el usuario actual
whatis : consulta sobre ~~un archivo~~ un comando whatis cp : copy files and directories

NOTA:

Scanned by CamScanner

who : muestra quienes están trabajando con el sistema operativo en momento determinado

pag 76

Interrumpir un comando en ejecución (crtl +c)

whereis whereis -> conocer ubicación de un comando y su página

whereis cp : /bin/cp /usr/share/man/man1/cp.1.gz

NOTA:

Scanned by CamScanner

Sesion de trabajo

PS (lista de procesos q' estan activos al momento de ejecutar el comando) comando de transaccion

- -a (lista todos los procesos)
- -f (muestra las dependencias de procesos)
- -X (muestra los procesos sin terminal de control)
- -u (muestra el nombre de usuario q' inicio el proceso, su hora de comienzo y otros datos)

Comandos del Sistema operativo

wc -? n° de lineas - palabras - bytes (cuenta lineas, palabras y caracteres de los archivos nombrados)

wc (opcion) (nombre_archivo)

- -l (cuenta solo la cantidad de lineas)
- -w (" " " " " palabras
- -c (" " " " " caracteres)

NOTA:

Scanned by CamScanner

Programacion orientada a objetos

- Es una tecnica de programacion q' hace hincapie en la naturaleza de los Objetos, q' estan sujetos a un determinado proceso.
- Se basa en el encapsulamiento de objetos y procesos asociados en un modulo llamado "clase", el cual ha sido creado a partir de una struct

Clase: conjunto de datos completamente definidos por el usuario asociados a funciones y operadores validos unicamente para ellos

clases y miembros

- El acceso a los objetos de una clase puede ser restringido a un par de funciones declaradas como parte de la clase.
- Esto se denominan metodos y pueden ser "miembros" o "funciones amigas"
- los objetos de una clase se crean con funciones miembros llamadas "constructores" y en algunos casos requieren ser destruidos

```
Ej    class par          Atr inic private:
      {     int x; int y;
                                          destructor  lleva el mismo nombre q' el constructor
            Public:
                  Par ( ) { }
                  int suma ( );   ~complejo ( );      // constructor, lleva el mismo nombre q' la clase (no puede devolver nada
      };
      int main ( )
      { class par p1 ( );   -   class par = p1 ( );
        p1.suma ( )                                   // destructor
es un metodo

      int par :: suma ( )                             // declaro fncion
      { }

      complejo :: ~complejo ( )
      { cout << "destructor";
      }
```

- Por defecto en una clase los elementos son privados (int x, char, doble) por eso no escribimos "private"

Scanned by CamScanner

Sobrecarga de funciones

- las funciones con el mismo nombre se denominan sobrecarga.
- Difieren entre sí por el nº de argumentos
- la misma función puede realizar tareas distintas

```
class suma
{
    public:
        suma ( );
        int suma1 ( int, int );
        int suma1 ( int, int, int );
};

int suma :: suma1 (int n1, int n2)
{ return (n1+n2);
}

int suma :: suma1 ( int n1, int n2, int n3)
{ return (n1+n2+n3);
}

int main ()
{   class suma = s;
    int a=1, b=2, c=3;
    cout << "la suma de los 2 nº" << s.suma1(a,b); imprimirá 3
    cout << "la suma de los 3 nº"; << s.suma1 (a,b,c);
    imprimirá 5
```

sobrecarga de funciones constructoras

Tiene el mismo significado q' sobrecarga de funciones

NOTA

Herencia

- Nos permite reutilizar el codigo de proyectos previos
- Implica una "especialización," ya q' conservamos todas las prop. de la clase original, agregando particularidades de la nueva clase q' estamos definiendo

" los objetos creados a partir de estas clases, heredan datos y funciones de la clase base

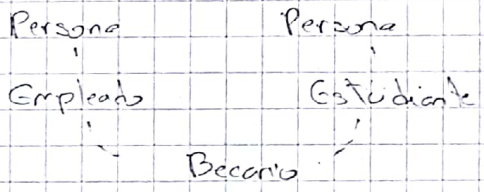
~~Herencia multiple~~

- Es un mecanismo para agregar a una clase existente nuevas prestaciones sin necesidad de reprogramar

Herencia multiple

- Permite la creacion de clases derivadas a partir de varias clases base

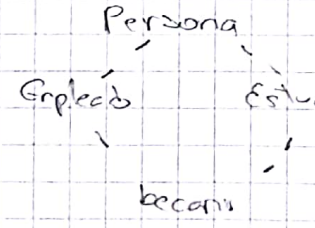
Herencia virtual



la clase becario hereda 2 veces los datos y func de "persona" esto provoca ambigüedad

. Es x esta razon q' utilizamos herencia virtual

```
class Estudiante : virtual public persona;
class Empleado : virtual public persona
class becario: public Estudiante, public Empleado
```



Scanned by CamScanner

Polimorfismo

(Hay la posibilidad de tener muchas formas)

Es: Permite tratar a los objetos derivados como si fueran objeto base

Se refiere a la propiedad por la cual es posible enviar mensajes ~~similares~~ practicamente iguales a objetos de ~~distintos~~ distintos. El unico requisito al deban cumplir los objetos; es saber responder al mensaje al se envia.

NOTA:

Scanned by CamScanner

Destructores

- Es un metodo ~~que todo existe un~~ q' actua por defecto al definirse un objeto
- Una de sus utilidades es la recuperacion de memoria de objetos q' utilizan memoria dinamica
- llevan el mismo nombre q' la clase precedido de un Tilde (~)

```
class suma
{   public
        suma ()
        ~suma ().
};
```

Funciones Amigas

- Tienen acceso a los miembros privados de una clase, sin ser un metodo de la clase

friend tipo-de-funcion nombre (nom-clase ;)

```
class cont
{       friend void mostrar (cont &);
        int x=0
        Public:
                cont () { cout<< x } → Si imprimo mostraria 0
};

void mostrar (cont &c)
{   int num=2
        c.x = num → ahora va a mostrar 2 por q' accedio al elemento privado
}

int main()
{       class cont a;
        mostrar (a) → le paso el nombre de la clase numero.
}
```

NOTA:

Scanned by CamScanner

Clases Amigas

- Permite la comunicación entre clases, comparten los datos privados (similar a herencia, pero ahí la 2 clases pueden acceder a los datos privados)

funciones virtuales

Al crear una función virtual en la clase base, le damos permiso a una clase derivada, en poder modificar esa función

```
class base
{   Public
        virtual void dice() { cout << "es la clase base"}
}
class der : public base
{
    Public
        void dice (){ cout << es la clase derived
}

int main
{   class base *p;
    class der dv1;

    p= & dv1;
    (*p) dice();   -> mostrara es la clase derivada
}

int mar
{   clase base p
    class der dv1;

    p.dice();  // mostrara es la clase base
}
```

Clases abstractas

No tienen sentido, sirven para crear funciones virtuales, q van a ser usadas por clases derivadas.

NOTA:

Scanned by CamScanner

Pasar un vector a la función

s Clase
rad ingresar (int c[]);

A main
for (i=0; i<tam; i++)
{ cin>> c[i];
}
b. ingresar (c) // así le paso todo lo q' contiene el vector

Ingresar un nombre

1) # include <string>
class... string nom[tam] → en el caso de varios nombres

función... getline (cin, nom[i]);

Limpar pantalla

include <stdlib.h>
system ("/usr/bin/clear");

Generar n° aleatorio

#include <stdlib.h>

int b, c; Srand (Time(null));
b = rand();
b = abs (b); // obligo a Valor abs a sea (+)
c = (int) b % 4; → n° tope
a = c;
variable de la clase

Entre límites

c = Li + rand() % Ls;

80 - 39

c = rand() % 60 + 20

va a generar un n° del 1-9 y luego le sumamos 20 para q' esté en el int

NOTA:

Scanned by CamScanner

Copiar nombre

```
#include <cstring>
void Persona::ingresa(char* n)
strcpy(nombre, n)
```

Main

```
char nombre[tam]

cout << ...        cin >> nombre   // así se ingresa
```

NOTA:

Scanned by CamScanner